CS102 – Computer Programming I

Lecture 01: Introduction

New Uzbekistan University (NewUU)

Summer 2026

Today's roadmap

Course Overview & Logistics

What is a Computer?

Elements of a Computer System

Number Systems & Data Representation

Programming Languages

Processing a C++ Program

History of C and C++

Your First C++ Program

Summary & Practice

Course Overview & Logistics

What this course is about

- **CS102 – Computer Programming I** teaches you to think like a programmer and to express solutions in **C++**.
- By the end of the course you will be able to:
  - break a problem into clear, logical steps (algorithms);
  - read, write, compile, and debug C++ programs;
  - use selection, repetition, functions, arrays, and pointers.
- **No prior programming experience is assumed** – we start from zero.

Mindset for this course

Programming is learned *by doing*. Reading slides is not enough – you must write, run, and break code every week.

What you need

Tools

- A C++ compiler (e.g. `g++` / GCC, Clang, or MSVC).
- A code editor or IDE (VS Code, Code::Blocks, CLion).
- Or simply an **online compiler** – no install needed.

Habits

- Attend every lecture and lab.
- Type out every example yourself.
- Start assignments early.
- Ask questions – in class and in office hours.

How a typical day works

- **Lecture (2h)** – new concepts, worked examples, live coding.
- **Lab / practice (2h)** – you write code and get help on the spot.
- **Quiz** – short 15-minute check at the start of selected days.

This is an *intensive* summer course: one full topic every day. Each topic builds on the previous one – falling behind even a single day is the biggest risk, so keep up daily.

Grading criteria

Assessment

Component	Weight
Quizzes (4 ×)	30 %
Final exam	70 %
Total	**100 %**

No midterm and no assignments this term.

Attendance

Minimum **75 %** attendance is required to sit the final exam.

Final = 70 %

Most of your grade is on the final exam, so master the fundamentals every day. The four quizzes (30 %) keep you on track.

Academic integrity

- Discussing ideas is encouraged; **copying code is not**.
- Submit work that is your own; cite any external help.
- Plagiarism and contract cheating carry serious penalties.

The honest truth

The final exam is 70 % of your grade and you sit it alone. Relying on someone else during quizzes only guarantees you will struggle on the final.

Course Learning Outcomes (CLOs)

CLO1 Understand basic problem-solving steps and logic constructs.

CLO2 Apply core programming concepts using *sequence*, *selection*, and *repetition* structures.

CLO3 Design and implement algorithms in C++ to solve real-world problems.

Everything we do this semester maps back to one of these three outcomes.

Topics we will cover

1. Introduction **(today)**
2. Algorithms
3. Basic components of a C++ program
4. Operators & conditional statements
5. Loops
6. Functions
7. Recursion
8. Arrays I
9. Arrays II
10. Pointers I
11. Pointers II
12. File handling
13. Structures

Today's goal: understand *what a computer is* and *how a program becomes something the machine can run*.

What is a Computer?

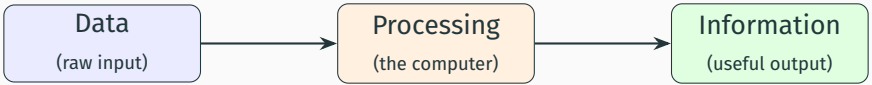
What is a computer?

- The word *computer* comes from *compute* – “to calculate”.
- A **computer** is an electronic device that performs arithmetic and logic operations at very high speed.
- It is also called a **data processor**.

At its heart, a computer does only three things

Input $\rightarrow$ **Process & Store** $\rightarrow$ **Output**

Data processing: data vs. information



- **Data** is the raw material used as input (e.g. a list of exam marks).
- **Information** is the processed result (e.g. the class average).

Categories of computers

Mainframe computers Huge machines used by large organisations (banks, airlines, governments).

Midsize / server computers Shared, multi-user machines that serve many clients.

Microcomputers Personal computers, laptops, tablets, and phones.

Note (2026)

The phone in your pocket is millions of times faster than the room-sized mainframes of the 1960s – yet it shares the same fundamental architecture we study today.

Elements of a Computer System

Two pillars: hardware and software

Hardware (physical)

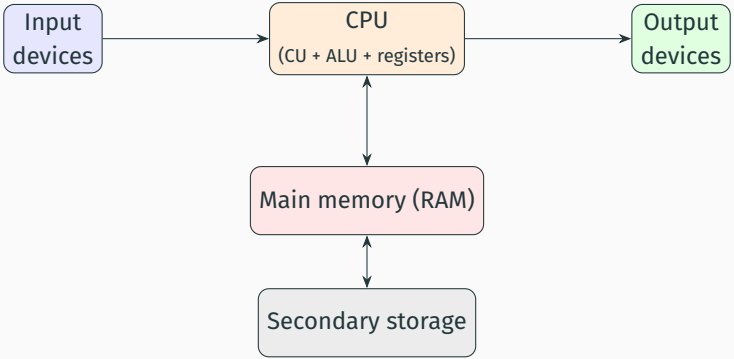
- Central Processing Unit (CPU)
- Main memory (RAM)
- Secondary storage
- Input / Output devices

Software (programs)

- System software (OS, drivers)
- Application software

Hardware is what you can touch; software is the set of instructions that tells the hardware what to do.

Block diagram of a computer



The CPU exchanges data with memory; memory exchanges data with storage; input flows in and output flows out.

The CPU – “brain” of the computer

The CPU has three main parts:

Control Unit (CU) Directs operations; tells memory, the ALU, and the I/O devices how to respond to program instructions.

Arithmetic & Logic Unit (ALU) Performs arithmetic (add, subtract, …) and logic (AND, OR, NOT, comparisons).

Registers Tiny, very fast storage locations inside the CPU.

We look at each part in turn.

CPU 1: the Control Unit (CU)

- A control unit is the circuitry that **directs operations** within the processor.
- It interprets each instruction and signals the other parts – the ALU, memory, and I/O devices – on how to respond.
- Think of it as the “traffic controller” of the CPU.

CPU 2: the Arithmetic & Logic Unit (ALU)

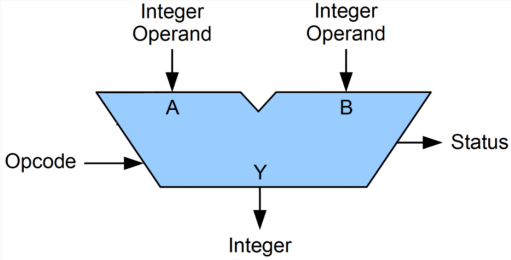
The ALU carries out the actual computation.

Arithmetic operations

- Addition: $Y = A + B$
- Subtraction: $Y = A - B$

Logic operations

- Bitwise **AND** / **OR** of A, B
- Comparisons ($<$, $=$, $>$)



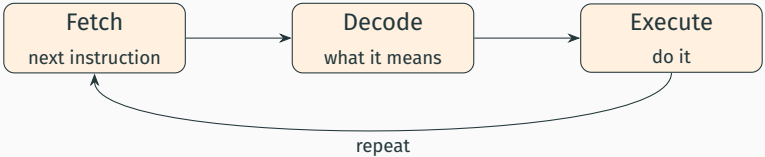
Operands A, B and an opcode go in; result Y and status come out.

CPU 3: registers

- Registers are the smallest, fastest storage in the computer – located *inside* the CPU.
- Two important ones:

  PC – program counter points to the *next* instruction to be executed.
  IR – instruction register holds the instruction *currently* being executed.

How the CPU runs a program: fetch–decode–execute



1. **Fetch** the instruction the PC points to.
2. **Decode** it (the CU works out what to do).
3. **Execute** it (often using the ALU); update the PC.

The CPU repeats this billions of times per second.

Main memory (RAM)

- Directly connected to the CPU.
- Every program must be loaded into main memory before it can run, and all data must be brought here before it can be manipulated.
- Volatile: when the power is turned off, everything in main memory is lost.

Why this matters to you

A running program lives in RAM. If you do not *save* your work to storage, a power cut erases it.

Memory as a row of numbered boxes



- Memory is a long sequence of **cells**, each with a unique numeric **address**.
- A *variable* (which we meet soon) is just a named box in memory.

Secondary storage

- A device that stores information **permanently** (non-volatile).
- Examples: solid-state drives (SSDs), hard disks, USB flash drives, optical discs.

Memory vs. storage

	Main memory (RAM)	Secondary storage
Speed	very fast	slower
Volatile?	yes (lost on power off)	no (permanent)
Used for	running programs	keeping files

Input / output devices

Input	Output
Keyboard, mouse, touchscreen, microphone, camera, secondary storage.	Monitor, printer, speakers, secondary storage.

Secondary storage appears in both lists: it can feed data in *and* receive results.

Software: system vs. application

Software is the set of instructions that controls the computer.

System software Controls the computer system itself – above all the *operating system* (Windows, macOS, Linux, UNIX), which monitors activity and provides services to other programs.

Application software Performs a specific task for the user: word processors, spreadsheets, browsers, games, mobile apps.

The programs *you* will write in CS102 are application software.

Number Systems & Data Representation

Why binary?

- Inside the computer, everything is electrical signals: **on** or **off**.
- We represent these two states with the digits **1** and **0** – the *binary* number system.
- All data – numbers, text, images, sound – is ultimately stored as patterns of 0s and 1s.

Vocabulary

Bit	a single binary digit, 0 or 1
Byte	a group of 8 bits

Reminder: how decimal (base 10) works

Each digit's value depends on its *position* (a power of 10):

$$\underbrace{2}_{\times 10^2}\ \underbrace{0}_{\times 10^1}\ \underbrace{5}_{\times 10^0} = 2(100) + 0(10) + 5(1) = 205$$

- 10 symbols: $0, 1, 2, \ldots, 9$.
- Move one place left $\Rightarrow$ multiply the place value by 10.

Binary (base 2) works the same way

Now the place values are powers of **2**:

$$\underbrace{1}_{\times 2^3}\ \underbrace{1}_{\times 2^2}\ \underbrace{0}_{\times 2^1}\ \underbrace{1}_{\times 2^0} = 8 + 4 + 0 + 1 = 13$$

2^3	2^2	2^1	2^0
8	4	2	1
1	1	0	1

So binary `1101` = decimal **13**.

Decimal $\rightarrow$ binary: repeated division by 2

Convert **13** to binary – divide by 2, keep the remainders:

$13 \div 2$	$= 6$	remainder **1**
$6 \div 2$	$= 3$	remainder **0**
$3 \div 2$	$= 1$	remainder **1**
$1 \div 2$	$= 0$	remainder **1**

Read the remainders **bottom-to-top**: `1101`. ✓

How much can a byte hold?

- 1 bit $\rightarrow$ 2 values (0, 1).
- 2 bits $\rightarrow$ 4 values (00, 01, 10, 11).
- n bits $\rightarrow 2^n$ values.
- **1 byte = 8 bits** $\rightarrow 2^8 =$ **256** values (e.g. 0–255).

TABLE 1-1 Binary Units

Unit	Symbol	Bits/Bytes
Byte		8 bits
Kilobyte	KB	2^{10} bytes = 1,024 bytes
Megabyte	MB	1024 KB = 2^{10} KB = 2^{20} bytes = 1,048,576 bytes
Gigabyte	GB	1024 MB = 2^{10} MB = 2^{30} bytes = 1,073,741,824 bytes
Terabyte	TB	1024 GB = 2^{10} GB = 2^{40} bytes = 1,099,511,627,776 bytes

Representing text: ASCII

- Computers store letters as **numbers** using a code such as **ASCII**.
- Each character gets a number, then stored in binary.

Character	ASCII (decimal)	Binary (8 bits)
`'A'`	65	01000001
`'B'`	66	01000010
`'a'`	97	01100001
`'0'`	48	00110000

So the text you type is really just numbers in disguise.

In-class exercise: number systems

In-class exercise

1. Convert binary `1010` to decimal.
2. Convert decimal `20` to binary.
3. How many distinct values can `4` bits represent?
4. Using the ASCII table, what binary stores the letter `'B'`?

(Answers: `10`; `10100`; `16`; `01000010`.)

Programming Languages

Machine language: the computer's native tongue

- A computer ultimately understands only **machine language** – instructions encoded as 0s and 1s.
- It is fast for the machine but extremely hard for humans to read, write, or debug.
- So we invented friendlier languages and tools to translate them.

Low-level vs. high-level languages

Low-level Machine language and assembly language – close to the hardware.

High-level C, C++, Java, Python, JavaScript – close to human language; far easier to read and write.

Whatever language we use, the code is ultimately translated into machine language so the CPU can run it.

The same idea at three levels

Compute `wages = rate × hours`.

Machine language (conceptual):

```
100100 010001 ; Load rate
100110 010010 ; Multiply by hours
100010 010011 ; Store the result
```

Assembly (mnemonics): `LOAD`, `MUL`, `STORE` …

C++ (high level):

```
1 wages = rate * hours;
```

Assembly mnemonics map to machine codes

TABLE 1-2 Examples of Instructions in Assembly Language and Machine Language

Assembly Language	Machine Language
LOAD	100100
STOR	100010
MULT	100110
ADD	100101
SUB	100011

Each human-readable mnemonic (LOAD, ADD, …) corresponds to a fixed pattern of bits the CPU understands.

Translators: compiler vs. interpreter

Compiler (C, C++)

Translates the *whole* program into machine code *once*, producing an executable that runs fast.

Interpreter (e.g. Python)

Translates and runs the program *line by line* each time it executes.

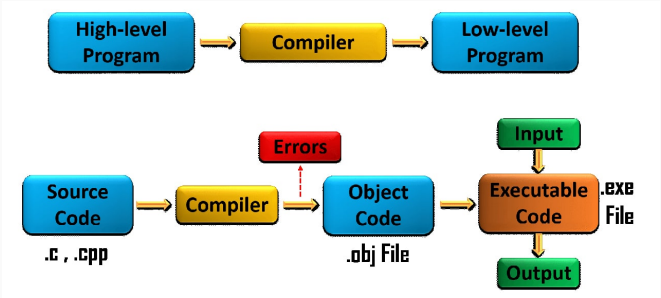
C++ is a **compiled** language – which is why we will talk about “building” our programs.

Key terms: source, object, executable

Source code The statements you write, saved in a source file (e.g. `main.cpp`).

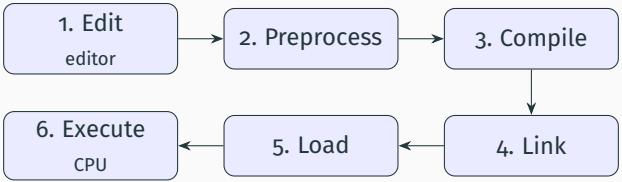
Object code Machine-language output produced by the compiler for each source file (`.obj`).

Executable The final runnable program, after *linking* object code with libraries (`.exe`).



Processing a C++ Program

Phases of building a C++ program



Edit → Preprocess → Compile → Link → Load → Execute.

Each phase, explained

1. **Edit:** write source code in an editor; save to disk.
2. **Preprocess:** handle directives such as `#include`.
3. **Compile:** turn source code into object code.
4. **Link:** combine object code with libraries to make an executable.
5. **Load:** the loader places the program into main memory.
6. **Execute:** the CPU runs the instructions one by one.

Building from the command line

With the GCC compiler, the whole pipeline is one command:

```
# compile main.cpp into an executable called "program"
g++ main.cpp -o program

# run it
./program
```

- `g++` compiles *and* links in one step.
- `-o program` names the output file.

Setting up your environment

Option A – install locally

Install a compiler (GCC/MinGW, Clang) and an editor/IDE (VS Code, Code::Blocks, CLion).

Option B – use an online compiler (fastest to start)

Sites such as *OnlineGDB*, *Programiz*, or *Compiler Explorer* let you write and run C++ in the browser – nothing to install.

For the first weeks, an online compiler is perfectly fine.

History of C and C++

A short history

C

- Evolved from BCPL and B (“typeless” languages).
- **Dennis Ritchie**, Bell Labs, 1970s – added data types.
- Grew alongside the UNIX operating system.
- Standardised: ANSI C (1989), ISO C (1990).

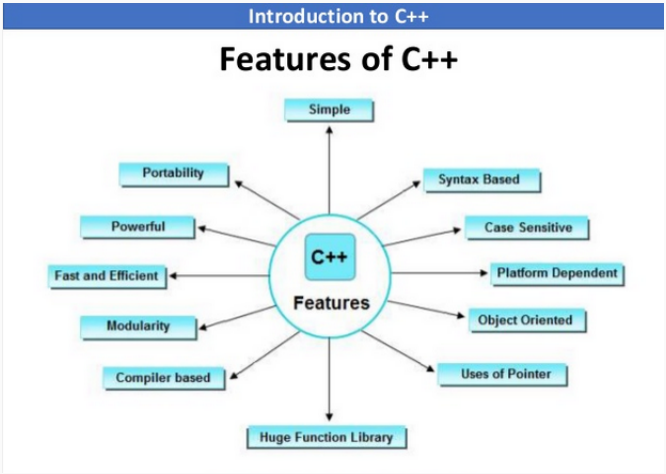
C++

- **Bjarne Stroustrup**, Bell Labs, early 1980s.
- Added object-oriented programming to C.
- Programs are built from *classes* and *functions*.
- Rich C++ standard library.

Why learn C++?

- **Strong foundation** – variables, types, control structures, functions, arrays, pointers, memory management.
- **Procedural and object-oriented** – start simple, grow into classes, inheritance, polymorphism.
- **Performance & memory insight** – see how programs use hardware.
- **Industry-relevant** – operating systems, game engines, embedded and high-performance systems.
- **Transferable** – Java, C#, Python, … feel familiar afterwards.

Features of C++ at a glance



Your First C++ Program

“Hello, World!”

```cpp
#include <iostream>
using namespace std;

int main() {
   cout << "Hello, World!" << endl;
   return 0;
}
```

By tradition, the first program in any language just prints a greeting. Let us dissect it.

Anatomy of the program

```cpp
#include <iostream> // 1. bring in input/output features
using namespace std; // 2. so we can write cout, not std::cout

int main() { // 3. execution always starts here
    cout << "Hello, World!" << endl; // 4. print + newline
    return 0; // 5. tell the OS: success
}
```

- `main()` is the entry point – every program has exactly one.
- `cout <<` sends data to the screen; `endl` ends the line.
- Statements end with a semicolon `;`.

Variables: a bigger example

```cpp
#include <iostream>
using namespace std;

int main() {
   double rate, hours;
   cout << "Enter hourly rate: ";
   cin >> rate;
   cout << "Enter hours worked: ";
   cin >> hours;

   double wages = rate * hours; // the formula from earlier
   cout << "Wages = " << wages << endl;
   return 0;
}
```

This is the `wages = rate * hours` idea as a real, compilable program – and a preview of next week's topic, *variables*.

Common beginner mistakes

- Forgetting the semicolon `;` at the end of a statement.
- Mismatched braces `{ }` or parentheses `( )`.
- Misspelling names – C++ is **case-sensitive** (`Cout` $\neq$ `cout`).
- Forgetting `#include <iostream>` before using `cout`.

Good news

The compiler reports these errors with a line number – learning to read error messages is a core skill.

Summary & Practice

Summary

- A computer is a fast electronic data processor (hardware + software).
- The CPU (CU, ALU, registers) repeats fetch–decode–execute; memory, storage, and I/O devices support it.
- All data is stored in **binary**; a byte is 8 bits and holds 256 values.
- Computers run machine language; we write high-level C++ and a *compiler* translates it through edit$\rightarrow\ldots\rightarrow$execute.
- C++ gives a strong, transferable foundation – and we already wrote and dissected our first program!

Practice questions

1. Name the three components of the CPU and state what each does.
2. Why is main memory (RAM) called *volatile*? Give one consequence for a programmer.
3. Convert binary `1101` to decimal and decimal `20` to binary.
4. How many distinct values can one byte represent? Why?
5. Put these in order: Compile, Edit, Execute, Link, Load, Preprocess.
6. Explain the difference between *source code*, *object code*, and an *executable*.
7. **Hands-on:** modify “Hello, World!” to also print your name on a second line; compile and run it.

Before next time

- Get a working C++ setup (local compiler *or* an online one).
- Type, compile, and run the “Hello, World!” program yourself.
- Try the hands-on practice question above.
- **Next lecture:** *Algorithms* – how we plan a solution before writing any code.

Thank you!

Questions?